

PREDICTFUSE

Final Project Definition & Implementation Roadmap

Predict → Prevent → Verify

An AI-powered pharmaceutical cold-chain safety system that predicts thermal excursions before critical exposure, autonomously intervenes, and verifies exposure at the individual-vial level using passive fuse indicators.

This document answers	What we are building
Hardware	What we will physically build and the order we will build it
Software	What the software must do and the order we will implement it
Team	A simple 4-person software division with clear ownership
Final demo	Exactly how the complete system should behave

This roadmap is the single implementation document for the project. It intentionally focuses on the final system and the build plan rather than comparing the two source documents.

1. What We Are Building

PredictFuse is a smart pharmaceutical cold-chain box/system. It continuously observes the shipment environment, predicts whether a thermal excursion is developing, decides what intervention is required, physically acts on the shipment, and records which individual vials actually experienced a validated exposure threshold.

The key idea is simple:

```

SENSE
↓
PREDICT
↓
DECIDE
↓
ACT
↓
VERIFY
↓
EXPLAIN + DISPLAY

```

Prediction is not verification. Temperature sensors and the prediction engine tell us where the shipment is heading. The passive fuse card gives physical evidence that a particular vial crossed a defined thermal-exposure level. The source concept explicitly makes this distinction. [filecite]turn0file0[L5-L24]

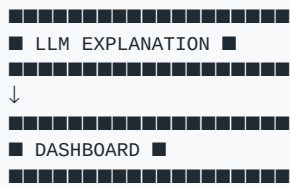
The system therefore has two complementary kinds of evidence:

- Continuous digital evidence: temperature, humidity, GPS, forecasts, decisions and actuator events.
- Physical vial-level evidence: passive fuse-card state changes associated with individual vials.

The final outcome is not just “temperature exceeded.” It is: the system predicted the problem, attempted to prevent it, and can later identify which vial actually crossed an exposure threshold.

2. Final System Architecture

[illegible]



The Smart Spine is the reusable electronic controller. The source architecture assigns the ESP32 sensor reading, fuse-state checking, timestamps, local actuation, buffering and communication responsibilities.

[filecite\turn0file0\L38-L52](#)

3. Hardware We Will Build

The hardware should be developed in layers. Do not attempt the entire box at once.

Hardware	Purpose	Build stage
ESP32	Main controller and edge computer	Stage 1
Temperature sensor	Measure shipment environment	Stage 1
Humidity sensor	Environmental context	Stage 1
GPS module	Location and route context	Stage 2
Fuse-card reader	Detect passive fuse state changes	Stage 3
Passive fuse cards	Vial-level thermal exposure evidence	Stage 3
Peltier module	Prototype active cooling	Stage 4
MOSFET/driver	Safe electronic switching of Peltier	Stage 4
Servo	Prototype vent/airflow control	Stage 4
LEDs/buzzer	Local system status and alarm	Stage 4
Power system	Supply controller and actuators	Stage 4
Insulated box	Physical cold-chain test environment	Stage 5

The source concept proposes ESP32, temperature/humidity sensing, GPS, fuse-card continuity reading, communications, Peltier cooling, servo control and LEDs/buzzer as the prototype hardware elements.

[filecite\turn0file0\L38-L62](#)

4. Hardware Development Roadmap

Stage H1 — ESP32 + environmental sensing

- Connect ESP32 to the temperature/humidity sensor.
- Read values every fixed interval.
- Add timestamps.
- Print readings to serial monitor.
- Validate sensor readings before connecting anything else.

Deliverable: stable environmental telemetry from ESP32.

Stage H2 — GPS

- Connect GPS module.
- Parse latitude/longitude.
- Attach location to telemetry.
- Test GPS loss and recovery.

Deliverable: telemetry contains temperature, humidity and location.

Stage H3 — Passive fuse system

- Create a prototype passive fuse card with defined demonstration states.
- Map every card to a vial ID.
- Build continuity/state-reading circuitry.
- Record state transitions with timestamp and zone.
- Test multiple fuse levels.

The fuse thresholds must be experimentally validated for the target pharmaceutical; they should not be presented as arbitrary universal drug-stability thresholds. [filecite]turn0file0[L53-L59]

Stage H4 — Actuation

- Connect Peltier through an appropriate driver.
- Connect servo for vent control.
- Add LEDs and buzzer.
- Implement local emergency safety behavior.

Deliverable: ESP32 can execute a received cooling/vent/alert command.

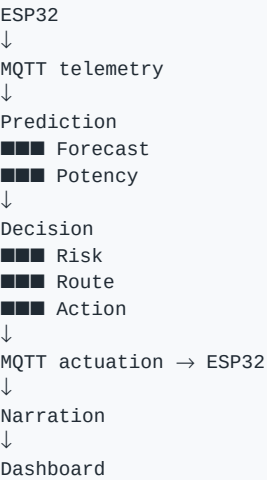
Stage H5 — Physical cold-chain box

- Install sensors in representative positions.
- Install fuse-card/vial zones.
- Install cooling hardware safely.
- Run controlled warming/cooling experiments.

Deliverable: a repeatable physical prototype capable of demonstrating the complete loop.

5. Software We Will Build

The software will sit above the ESP32 firmware. Its job is to transform raw telemetry into prediction, decision, physical commands, explanations and visualization. ColdGuard defines this software boundary and the core loop as Predict → Decide → Act → Explain → Visualize. [filecite\[turn0file1L3-L20\]](#)



Recommended stack

Area	Technology
Firmware interface	ESP32 + MQTT
Messaging	MQTT / Mosquitto
Prediction	Python + NumPy/SciPy
ML enhancement	XGBoost after physics baseline
Backend	FastAPI
Database	PostgreSQL
Frontend	React + Tailwind
Charts	Recharts
Map	Leaflet or equivalent
Narration	LLM API + deterministic fallback

6. Software Interface — MQTT

Before coding, the team must freeze one shared data contract. ColdGuard explicitly recommends agreeing on the topic/schema table before implementation because changing output fields can break downstream services. [filecite\[turn0file1L262-L280\]](#)

Topic	Meaning
coldguard/telemetry	ESP32 → software: temperature, humidity, GPS, fuse states
coldguard/forecast	Prediction → decision/dashboard
coldguard/potency	Prediction → decision/dashboard
coldguard/decision	Decision → narration/dashboard
coldguard/actuate	Decision → ESP32
coldguard/narration	Narration → dashboard

```
Telemetry
{
  "timestamp": "...",
  "ambient": {
```

```

"temp_c": 6.2,
"humidity_pct": 41
},
"gps": {
"lat": 12.9716,
"lng": 77.5946
},
"zones": [
{
"zone_id": 2,
"fuse_triggered": false
}
]
}

```

7. Software Team Division — 4 People

Keep the division simple. Each person owns one major software block, but everyone works against the same MQTT contract.

Person	Owns	Main work	Final deliverable
Person 1	Prediction / AI	Simulator, thermal forecast, time-to-threshold, potency, later ML	Working prediction service
Person 2	Backend / Decision	MQTT integration, FastAPI, DB, risk, route and actuation logic	Decision + backend service
Person 3	Frontend / Dashboard	React UI, zones, charts, GPS/map, potency, fuse events	Live dashboard
Person 4	LLM / Integration / Testing	Narration, fallback, replay tests, integration, demo orchestration	Reliable end-to-end software loop

Important: Person 4 is not “just the LLM person.” Their main responsibility is making sure all four software components work together and the final demo is reliable.

Person 1 — Prediction / AI

- Build the telemetry simulator first.
- Generate normal, warming, sudden-spike and localized-excursion scenarios.
- Implement Newton's Law of Cooling.
- Fit a separate warming/cooling rate for each zone.
- Calculate time-to-threshold.
- Implement configurable potency estimation.
- Later add ML as a prediction enhancement.

ColdGuard proposes Newton's Law of Cooling with a per-zone fitted rate constant and per-zone forecast output. [filecite\turn0file1\L68-L77](#)

Person 2 — Backend / Decision

- Set up MQTT broker and subscriptions.
- Create FastAPI backend.
- Create PostgreSQL schema.
- Consume forecast and potency data.
- Calculate LOW/MEDIUM/HIGH/CRITICAL risk.
- Implement route comparison.
- Generate zone-specific actuation commands.
- Add cooldown/hysteresis to prevent rapid actuator toggling.

ColdGuard assigns route-aware decisions and zone-scoped actuation to this layer.

[filecite\turn0file1\L98-L158](#)

Person 3 — Frontend / Dashboard

- Build the UI with mock data first.
- Create shipment status header.
- Create zone/vial grid.
- Create live temperature forecast chart.
- Create time-to-threshold countdown.
- Create potency display.
- Create GPS/route map.
- Create fuse-event history.
- Create actuation status.
- Connect to live MQTT/WebSocket data after backend topics stabilize.

ColdGuard recommends a React dashboard with zone tiles, forecast curves, route map, potency gauge, narration feed and countdown. [filecite\turn0file1\L213-L261](#)

Person 4 — LLM / Integration / Testing

- Build the narration service from structured decision JSON.
- Ensure the LLM cannot invent values or issue physical commands.
- Create deterministic fallback narration.
- Create replay scripts that test the entire software chain.
- Test MQTT topic compatibility.
- Test network-loss behavior.
- Test one-zone excursion isolation.
- Own final integration and demo reliability.

ColdGuard specifically recommends structured JSON grounding, narration on meaningful state transitions, and a template fallback when the LLM/API is unavailable. [filecite\turn0file1\L178-L209](#)

8. Software Development Roadmap

The order below lets all four people work in parallel without creating dependency chaos.

Week/Stage	What happens	People
S0 — Day 1	Freeze MQTT schemas, zone/vial model, DB entities and repository	All 4
S1 — Days 1-3	Build simulator + MQTT + dashboard mock	P1 + P2 + P3
S2 — Days 3-6	Build physics prediction + backend risk logic	P1 + P2
S3 — Days 5-8	Build dashboard live integration	P3 + P2
S4 — Days 7-10	Add potency + route decision + actuation commands	P1 + P2
S5 — Days 9-12	LLM narration + fallback + event feed	P4 + P3
S6 — Days 11-14	Full replay testing + hardware integration	All 4
S7 — Final	Polish, failure testing and demo scenario	All 4

Stage S0 — Shared contract

- Create SYSTEM_CONTRACT.md.
- Freeze all MQTT topic names.
- Freeze JSON field names.
- Define zone_id and vial_id rules.
- Define risk levels and actuator commands.

Stage S1 — Simulation first

- Create telemetry generator.
- Create a normal shipment scenario.
- Create a localized Zone 2 warming scenario.
- Create a cooling-success scenario.
- Create a cooling-failure scenario.
- Make MQTT replayable.

This is critical: software development should not wait for the physical prototype. ColdGuard explicitly recommends developing prediction and decision against simulated/replayed telemetry.

📄filecite📄turn0file1📄L274-L277📄

Stage S2 — Prediction + risk

- Implement temperature smoothing.
- Fit per-zone thermal rate.
- Predict future temperature.
- Calculate time-to-threshold.
- Generate risk level.

Stage S3 — Dashboard

- Connect telemetry.
- Display each zone independently.
- Display forecast.
- Display countdown.
- Display current risk.

Stage S4 — Potency + decisions + actuation

- Add configurable potency estimate.
- Compare current route vs alternate route.

- Generate decision object.
- Publish actuation command.
- Test command against simulated ESP32.

Stage S5 — Narration

- Generate explanation from decision JSON.
- Add fuse evidence to narration context.
- Trigger only on meaningful state changes.
- Add fallback templates.

Stage S6 — Full integration

```

Temperature rises
↓
Prediction detects risk
↓
Decision selects action
↓
MQTT publishes command
↓
ESP32 activates Peltier/servo
↓
Fuse state is monitored
↓
Dashboard updates
↓
Narration explains event

```

9. Hardware + Software Integration Order

Integration	Connect
I1	ESP32 → temperature sensor
I2	ESP32 → GPS
I3	ESP32 → MQTT
I4	Simulator → same MQTT telemetry topic
I5	Prediction → forecast/potency topics
I6	Decision → actuation topic
I7	ESP32 → physical Peltier/servo
I8	ESP32 → fuse reader
I9	Fuse events → dashboard
I10	LLM narration → final dashboard

At every stage, keep the simulator working. If hardware fails during a demonstration, the software should still be testable using replayed telemetry.

10. Final End-to-End Scenario

Step 1 — Normal

- Temperature stable.
- All fuse levels intact.
- Risk is LOW.
- Dashboard is green.

Step 2 — Thermal problem begins

- One zone begins warming faster than others.

- Prediction engine detects the trend.
- Time-to-threshold starts decreasing.

Step 3 — System predicts danger

Zone 2
 Current temperature: 9.1°C
 Projected temperature: 12.4°C
 Time to threshold: 210 seconds
 Risk: HIGH

Step 4 — System prevents exposure

- Decision engine activates cooling.
- Servo/vent is adjusted if required.
- Driver/operator receives an alert.
- Alternate route may be recommended.

Step 5 — Physical verification

- If a fuse threshold changes, the system records the associated vial.

Vial: V17
 Zone: 2
 Fuse level: 2
 Trigger time: 10:28:44
 Trigger temperature: 15.0°C

Step 6 — Recovery

- Cooling succeeds.
- Temperature decreases.
- Risk decreases.
- Higher fuse thresholds remain intact.

Step 7 — Explanation

The dashboard shows a concise system briefing explaining which zone was affected, how many vials were at risk, what action was taken, and what the physical fuse evidence shows.

11. Testing Plan

Test	Expected result
Normal telemetry	Risk remains LOW
Gradual warming	Forecast approaches threshold gradually
Localized warming	Only affected zone becomes high risk
Sudden spike	Risk escalates quickly
Cooling success	Temperature and risk fall
Cooling failure	Risk continues escalating and alerts remain active
Fuse trigger	Correct vial and fuse level are recorded
Network loss	Local safety behavior continues; data is buffered
LLM failure	Fallback narration is generated
Dashboard replay	All panels update correctly without changing unrelated zones
End-to-end	Predict → Decide → Act → Verify → Explain works

The localized-zone test is especially important because the architecture is intentionally spatial: a problem in one zone should not automatically change the state of every other zone. ColdGuard explicitly calls this out as a core testing requirement. [filecite\[turn0file1\[L94-L97\]](#)

12. What We Must Finish First

Do not try to finish every advanced feature before the core loop works.

Priority	Must have
1	ESP32 + temperature sensing
2	MQTT telemetry
3	Simulator
4	Thermal prediction + time-to-threshold
5	Risk/decision engine
6	Peltier/servo actuation
7	Fuse-card state detection or prototype simulation
8	Dashboard
9	End-to-end replay
10	Physical demonstration

After the MVP works: add potency estimation, better fuse materials/validation, ML forecasting, route optimization, LLM narration, cellular connectivity and advanced analytics.

13. Final Project Definition

We are building a predictive and autonomous pharmaceutical cold-chain safety system. The system uses environmental sensing and GPS to understand the shipment, physics/AI to predict thermal risk, a decision engine to choose an intervention, an ESP32 Smart Spine to execute the action, and passive fuse cards to verify actual exposure at the vial level.

The software team will be four people:

PERSON 1 → Prediction / AI
PERSON 2 → Backend / Decision
PERSON 3 → Frontend / Dashboard
PERSON 4 → LLM / Integration / Testing

The hardware team will build the Smart Spine, environmental sensing, GPS, passive fuse reader/cards, cooling, venting and local alerts in stages.

The final system must demonstrate one complete story:

Predict the problem → intervene before critical exposure → verify what actually happened → explain it clearly.

That is the single goal around which the hardware roadmap, software roadmap and team division should be organized.

Validation note: Fuse thresholds, phase-change materials, pharmaceutical stability relationships and potency parameters must be experimentally validated for the target pharmaceutical before any real-world safety, release or clinical claim is made. PredictFuse explicitly identifies these as validation requirements rather than already-established release criteria. [filecite turn0file0L125-L131](#)